

Linux 源码 阅读

- 如何获取源代码
- 如何编译源代码
- 如何更新系统内核
- 以文件系统为例分析源码

获取源码

- \$ `uname -rs` #了解当前OS的内核版本信息

`Linux 4.4.0-201-generic`

- \$ `apt search linux-source`

`Linux kernel source for version 4.4.0 with Ubuntu patches`

- \$ `sudo apt install linux-source-X.X.X`

`/usr/src/linux-source-4.4.0`

获取源码（其他方式）

- 源文件下载

- <https://mirrors.edge.kernel.org/pub/linux/kernel/>

- Git Clone

- <https://git.kernel.org/>
 - <https://github.com/torvalds/linux.git>

获取源码

● `/usr/src/linux-source-X.X.X$ ls`

`debian` `debian.master` `linux-source-4.4.0.tar.bz2`



ubuntu 发行版中针对内核
修改的配置、补丁、说明和
一些工具

linux 内核源码压缩包

获取源码

- `~$ tar -xjf /usr/src/linux-source-X.X.X.tar.bz2`
- `~/linux-source-X.X.X $ ls`

arch

block

certs

COPYING

CREDITS

crypto

Documentation

drivers

dropped.txt

firmware

fs

include

init

ipc

Kbuild

Kconfig

kernel

lib

MAINTAINERS

Makefile

mm

net

README

REPORTING-BUGS

samples

scripts

security

snapcraft.yaml

sound

spl

tools

ubuntu

usr

virt

zfs

获取源码

文件夹	描述
arch	包含了 Linux 内核的不同体系结构的实现，例如 x86、ARM、MIPS 等
block	包含了块设备驱动程序的实现，例如硬盘、闪存、CD-ROM 等
crypto	包含了加密和解密算法的实现，例如 AES、SHA1、SHA256 等
drivers	包含了各种设备的驱动程序，例如网络设备、USB 设备、声卡、显卡等
fs	包含了文件系统的实现，例如 ext4、Btrfs、FAT 等
include	包含了 Linux 内核的头文件，这些头文件包含了函数和数据结构的声明和定义
init	包含了 Linux 内核启动时的初始化程序，例如初始化内存管理、进程管理等
ipc	包含了进程间通信（IPC）机制的实现，例如信号量、消息队列、共享内存等
kernel	包含了 Linux 内核的核心代码，例如进程管理、中断处理、定时器等
lib	包含了一些 Linux 内核使用的通用库和函数
mm	包含了内存管理的实现，例如页面分配、虚拟内存映射等
net	包含了网络协议栈的实现，例如 TCP/IP、UDP、ICMP 等
scripts	包含了一些工具和脚本，例如用于构建内核的 Makefile、Kconfig 等
security	包含了 Linux 内核的安全子系统，例如 SELinux、AppArmor 等
sound	包含了音频设备的驱动程序和 ALSA 子系统的实现
tools	包含了一些与 Linux 内核开发相关的工具，例如 perf、kvm、kprobes 等
usr	包含了用户空间工具和库的代码，例如内核模块加载器 modprobe、libc 库等

编译源码

- \$ make menuconfig

```
.config - Linux/x86 4.4.262 Kernel Configuration

Linux/x86 4.4.262 Kernel Configuration
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted letters are hotkeys.
Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend:
[*] built-in [ ] excluded <M> module <> module capable

[*] 64-bit kernel
  General setup --->
  [*] Enable loadable module support --->
  [*] Enable the block layer --->
    Processor type and features --->
    Power management and ACPI options --->
    Bus options (PCI etc.) --->
    Executable file formats / Emulations --->
  [*] Networking support --->
    Device Drivers --->
    Ubuntu Supplied Third-Party Device Drivers --->
    Firmware Drivers --->
    File systems --->
    Kernel hacking --->
    Security options --->
  *- Cryptographic API --->
  *- Virtualization --->
    Library routines --->

<Select> < Exit > < Help > < Save > < Load >
```

编译源码

- `$ make menuconfig`
 - 生成一个 `.config` 文件
- `$ make`

编译后会生成一个内核镜像文件，通常命名为 `vmlinux` / `vmlinuz` / `bzImage`。这个文件是可以直接在硬件上运行的，包含了整个内核的代码和数据。

```
#
# Automatically generated file; DO NOT EDIT.
# Linux/x86 4.4.262 Kernel Configuration
#
CONFIG_64BIT=y
CONFIG_X86_64=y
CONFIG_X86=y
CONFIG_INSTRUCTION_DECODER=y
CONFIG_PERF_EVENTS_INTEL_UNCORE=y
CONFIG_OUTPUT_FORMAT="elf64-x86-64"
CONFIG_ARCH_DEFCONFIG="arch/x86/configs/x86_64_defconfig"
CONFIG_LOCKDEP_SUPPORT=y
CONFIG_STACKTRACE_SUPPORT=y
CONFIG_HAVE_LATENCYTOP_SUPPORT=y
CONFIG_MMU=y
CONFIG_NEED_DMA_MAP_STATE=y
CONFIG_NEED_SG_DMA_LENGTH=y
CONFIG_GENERIC_ISA_DMA=y
CONFIG_GENERIC_BUG=y
CONFIG_GENERIC_BUG_RELATIVE_POINTERS=y
CONFIG_GENERIC_TIME=y
CONFIG_GENERIC_HWEIGHT=y
CONFIG_ARCH_MAY_HAVE_PC_FDC=y
CONFIG_X86_GDB_SCRIPTS=y
CONFIG_X86_TSC=y
CONFIG_X86_MCE=y
CONFIG_X86_MCE_P6_P45=y
CONFIG_X86_MCE_P45_P54=y
CONFIG_X86_MCE_P55_P60=y
CONFIG_X86_MCE_P62_P68=y
CONFIG_X86_MCE_P72_P77=y
CONFIG_X86_MCE_P82_P94=y
CONFIG_X86_MCE_P95_P100=y
CONFIG_X86_MCE_P101_P104=y
CONFIG_X86_MCE_P105_P110=y
CONFIG_X86_MCE_P111_P115=y
CONFIG_X86_MCE_P116_P120=y
CONFIG_X86_MCE_P121_P125=y
CONFIG_X86_MCE_P126_P129=y
CONFIG_X86_MCE_P130_P132=y
CONFIG_X86_MCE_P133_P135=y
CONFIG_X86_MCE_P136_P139=y
CONFIG_X86_MCE_P140_P142=y
CONFIG_X86_MCE_P143_P145=y
CONFIG_X86_MCE_P146_P148=y
CONFIG_X86_MCE_P149_P150=y
CONFIG_X86_MCE_P151_P153=y
CONFIG_X86_MCE_P154_P156=y
CONFIG_X86_MCE_P157_P159=y
CONFIG_X86_MCE_P160_P162=y
CONFIG_X86_MCE_P163_P165=y
CONFIG_X86_MCE_P166_P168=y
CONFIG_X86_MCE_P169_P170=y
CONFIG_X86_MCE_P171_P173=y
CONFIG_X86_MCE_P174_P175=y
CONFIG_X86_MCE_P176_P178=y
CONFIG_X86_MCE_P179_P181=y
CONFIG_X86_MCE_P182_P184=y
CONFIG_X86_MCE_P185_P188=y
CONFIG_X86_MCE_P189_P190=y
CONFIG_X86_MCE_P191_P193=y
CONFIG_X86_MCE_P194_P195=y
CONFIG_X86_MCE_P196_P198=y
CONFIG_X86_MCE_P199_P200=y
CONFIG_X86_MCE_P201_P203=y
CONFIG_X86_MCE_P204_P206=y
CONFIG_X86_MCE_P207_P209=y
CONFIG_X86_MCE_P210_P212=y
CONFIG_X86_MCE_P213_P215=y
CONFIG_X86_MCE_P216_P218=y
CONFIG_X86_MCE_P219_P220=y
CONFIG_X86_MCE_P221_P223=y
CONFIG_X86_MCE_P224_P226=y
CONFIG_X86_MCE_P227_P229=y
CONFIG_X86_MCE_P230_P232=y
CONFIG_X86_MCE_P233_P235=y
CONFIG_X86_MCE_P236_P238=y
CONFIG_X86_MCE_P239_P240=y
CONFIG_X86_MCE_P241_P243=y
CONFIG_X86_MCE_P244_P246=y
CONFIG_X86_MCE_P247_P249=y
CONFIG_X86_MCE_P250_P252=y
CONFIG_X86_MCE_P253_P255=y
CONFIG_X86_MCE_P256_P258=y
CONFIG_X86_MCE_P259_P260=y
CONFIG_X86_MCE_P261_P263=y
CONFIG_X86_MCE_P264_P266=y
CONFIG_X86_MCE_P267_P269=y
CONFIG_X86_MCE_P270_P272=y
CONFIG_X86_MCE_P273_P275=y
CONFIG_X86_MCE_P276_P278=y
CONFIG_X86_MCE_P279_P280=y
CONFIG_X86_MCE_P281_P283=y
CONFIG_X86_MCE_P284_P286=y
CONFIG_X86_MCE_P287_P289=y
CONFIG_X86_MCE_P290_P292=y
CONFIG_X86_MCE_P293_P295=y
CONFIG_X86_MCE_P296_P298=y
CONFIG_X86_MCE_P299_P300=y
CONFIG_X86_MCE_P301_P303=y
CONFIG_X86_MCE_P304_P306=y
CONFIG_X86_MCE_P307_P309=y
CONFIG_X86_MCE_P310_P312=y
CONFIG_X86_MCE_P313_P315=y
CONFIG_X86_MCE_P316_P318=y
CONFIG_X86_MCE_P319_P320=y
CONFIG_X86_MCE_P321_P323=y
CONFIG_X86_MCE_P324_P326=y
CONFIG_X86_MCE_P327_P329=y
CONFIG_X86_MCE_P330_P332=y
CONFIG_X86_MCE_P333_P335=y
CONFIG_X86_MCE_P336_P338=y
CONFIG_X86_MCE_P339_P340=y
CONFIG_X86_MCE_P341_P343=y
CONFIG_X86_MCE_P344_P346=y
CONFIG_X86_MCE_P347_P349=y
CONFIG_X86_MCE_P350_P352=y
CONFIG_X86_MCE_P353_P355=y
CONFIG_X86_MCE_P356_P358=y
CONFIG_X86_MCE_P359_P360=y
CONFIG_X86_MCE_P361_P363=y
CONFIG_X86_MCE_P364_P366=y
CONFIG_X86_MCE_P367_P369=y
CONFIG_X86_MCE_P370_P372=y
CONFIG_X86_MCE_P373_P375=y
CONFIG_X86_MCE_P376_P378=y
CONFIG_X86_MCE_P379_P380=y
CONFIG_X86_MCE_P381_P383=y
CONFIG_X86_MCE_P384_P386=y
CONFIG_X86_MCE_P387_P389=y
CONFIG_X86_MCE_P390_P392=y
CONFIG_X86_MCE_P393_P395=y
CONFIG_X86_MCE_P396_P398=y
CONFIG_X86_MCE_P399_P400=y
CONFIG_X86_MCE_P401_P403=y
CONFIG_X86_MCE_P404_P406=y
CONFIG_X86_MCE_P407_P409=y
CONFIG_X86_MCE_P410_P412=y
CONFIG_X86_MCE_P413_P415=y
CONFIG_X86_MCE_P416_P418=y
CONFIG_X86_MCE_P419_P420=y
CONFIG_X86_MCE_P421_P423=y
CONFIG_X86_MCE_P424_P426=y
CONFIG_X86_MCE_P427_P429=y
CONFIG_X86_MCE_P430_P432=y
CONFIG_X86_MCE_P433_P435=y
CONFIG_X86_MCE_P436_P438=y
CONFIG_X86_MCE_P439_P440=y
CONFIG_X86_MCE_P441_P443=y
CONFIG_X86_MCE_P444_P446=y
CONFIG_X86_MCE_P447_P449=y
CONFIG_X86_MCE_P450_P452=y
CONFIG_X86_MCE_P453_P455=y
CONFIG_X86_MCE_P456_P458=y
CONFIG_X86_MCE_P459_P460=y
CONFIG_X86_MCE_P461_P463=y
CONFIG_X86_MCE_P464_P466=y
CONFIG_X86_MCE_P467_P469=y
CONFIG_X86_MCE_P470_P472=y
CONFIG_X86_MCE_P473_P475=y
CONFIG_X86_MCE_P476_P478=y
CONFIG_X86_MCE_P479_P480=y
CONFIG_X86_MCE_P481_P483=y
CONFIG_X86_MCE_P484_P486=y
CONFIG_X86_MCE_P487_P489=y
CONFIG_X86_MCE_P490_P492=y
CONFIG_X86_MCE_P493_P495=y
CONFIG_X86_MCE_P496_P498=y
CONFIG_X86_MCE_P499_P500=y
CONFIG_X86_MCE_P501_P503=y
CONFIG_X86_MCE_P504_P506=y
CONFIG_X86_MCE_P507_P509=y
CONFIG_X86_MCE_P510_P512=y
CONFIG_X86_MCE_P513_P515=y
CONFIG_X86_MCE_P516_P518=y
CONFIG_X86_MCE_P519_P520=y
CONFIG_X86_MCE_P521_P523=y
CONFIG_X86_MCE_P524_P526=y
CONFIG_X86_MCE_P527_P529=y
CONFIG_X86_MCE_P530_P532=y
CONFIG_X86_MCE_P533_P535=y
CONFIG_X86_MCE_P536_P538=y
CONFIG_X86_MCE_P539_P540=y
CONFIG_X86_MCE_P541_P543=y
CONFIG_X86_MCE_P544_P546=y
CONFIG_X86_MCE_P547_P549=y
CONFIG_X86_MCE_P550_P552=y
CONFIG_X86_MCE_P553_P555=y
CONFIG_X86_MCE_P556_P558=y
CONFIG_X86_MCE_P559_P560=y
CONFIG_X86_MCE_P561_P563=y
CONFIG_X86_MCE_P564_P566=y
CONFIG_X86_MCE_P567_P569=y
CONFIG_X86_MCE_P570_P572=y
CONFIG_X86_MCE_P573_P575=y
CONFIG_X86_MCE_P576_P578=y
CONFIG_X86_MCE_P579_P580=y
CONFIG_X86_MCE_P581_P583=y
CONFIG_X86_MCE_P584_P586=y
CONFIG_X86_MCE_P587_P589=y
CONFIG_X86_MCE_P590_P592=y
CONFIG_X86_MCE_P593_P595=y
CONFIG_X86_MCE_P596_P598=y
CONFIG_X86_MCE_P599_P600=y
CONFIG_X86_MCE_P601_P603=y
CONFIG_X86_MCE_P604_P606=y
CONFIG_X86_MCE_P607_P609=y
CONFIG_X86_MCE_P610_P612=y
CONFIG_X86_MCE_P613_P615=y
CONFIG_X86_MCE_P616_P618=y
CONFIG_X86_MCE_P619_P620=y
CONFIG_X86_MCE_P621_P623=y
CONFIG_X86_MCE_P624_P626=y
CONFIG_X86_MCE_P627_P629=y
CONFIG_X86_MCE_P630_P632=y
CONFIG_X86_MCE_P633_P635=y
CONFIG_X86_MCE_P636_P638=y
CONFIG_X86_MCE_P639_P640=y
CONFIG_X86_MCE_P641_P643=y
CONFIG_X86_MCE_P644_P646=y
CONFIG_X86_MCE_P647_P649=y
CONFIG_X86_MCE_P650_P652=y
CONFIG_X86_MCE_P653_P655=y
CONFIG_X86_MCE_P656_P658=y
CONFIG_X86_MCE_P659_P660=y
CONFIG_X86_MCE_P661_P663=y
CONFIG_X86_MCE_P664_P666=y
CONFIG_X86_MCE_P667_P669=y
CONFIG_X86_MCE_P670_P672=y
CONFIG_X86_MCE_P673_P675=y
CONFIG_X86_MCE_P676_P678=y
CONFIG_X86_MCE_P679_P680=y
CONFIG_X86_MCE_P681_P683=y
CONFIG_X86_MCE_P684_P686=y
CONFIG_X86_MCE_P687_P689=y
CONFIG_X86_MCE_P690_P692=y
CONFIG_X86_MCE_P693_P695=y
CONFIG_X86_MCE_P696_P698=y
CONFIG_X86_MCE_P699_P700=y
CONFIG_X86_MCE_P701_P703=y
CONFIG_X86_MCE_P704_P706=y
CONFIG_X86_MCE_P707_P709=y
CONFIG_X86_MCE_P710_P712=y
CONFIG_X86_MCE_P713_P715=y
CONFIG_X86_MCE_P716_P718=y
CONFIG_X86_MCE_P719_P720=y
CONFIG_X86_MCE_P721_P723=y
CONFIG_X86_MCE_P724_P726=y
CONFIG_X86_MCE_P727_P729=y
CONFIG_X86_MCE_P730_P732=y
CONFIG_X86_MCE_P733_P735=y
CONFIG_X86_MCE_P736_P738=y
CONFIG_X86_MCE_P739_P740=y
CONFIG_X86_MCE_P741_P743=y
CONFIG_X86_MCE_P744_P746=y
CONFIG_X86_MCE_P747_P749=y
CONFIG_X86_MCE_P750_P752=y
CONFIG_X86_MCE_P753_P755=y
CONFIG_X86_MCE_P756_P758=y
CONFIG_X86_MCE_P759_P760=y
CONFIG_X86_MCE_P761_P763=y
CONFIG_X86_MCE_P764_P766=y
CONFIG_X86_MCE_P767_P769=y
CONFIG_X86_MCE_P770_P772=y
CONFIG_X86_MCE_P773_P775=y
CONFIG_X86_MCE_P776_P778=y
CONFIG_X86_MCE_P779_P780=y
CONFIG_X86_MCE_P781_P783=y
CONFIG_X86_MCE_P784_P786=y
CONFIG_X86_MCE_P787_P789=y
CONFIG_X86_MCE_P790_P792=y
CONFIG_X86_MCE_P793_P795=y
CONFIG_X86_MCE_P796_P798=y
CONFIG_X86_MCE_P799_P800=y
CONFIG_X86_MCE_P801_P803=y
CONFIG_X86_MCE_P804_P806=y
CONFIG_X86_MCE_P807_P809=y
CONFIG_X86_MCE_P810_P812=y
CONFIG_X86_MCE_P813_P815=y
CONFIG_X86_MCE_P816_P818=y
CONFIG_X86_MCE_P819_P820=y
CONFIG_X86_MCE_P821_P823=y
CONFIG_X86_MCE_P824_P826=y
CONFIG_X86_MCE_P827_P829=y
CONFIG_X86_MCE_P830_P832=y
CONFIG_X86_MCE_P833_P835=y
CONFIG_X86_MCE_P836_P838=y
CONFIG_X86_MCE_P839_P840=y
CONFIG_X86_MCE_P841_P843=y
CONFIG_X86_MCE_P844_P846=y
CONFIG_X86_MCE_P847_P849=y
CONFIG_X86_MCE_P850_P852=y
CONFIG_X86_MCE_P853_P855=y
CONFIG_X86_MCE_P856_P858=y
CONFIG_X86_MCE_P859_P860=y
CONFIG_X86_MCE_P861_P863=y
CONFIG_X86_MCE_P864_P866=y
CONFIG_X86_MCE_P867_P869=y
CONFIG_X86_MCE_P870_P872=y
CONFIG_X86_MCE_P873_P875=y
CONFIG_X86_MCE_P876_P878=y
CONFIG_X86_MCE_P879_P880=y
CONFIG_X86_MCE_P881_P883=y
CONFIG_X86_MCE_P884_P886=y
CONFIG_X86_MCE_P887_P889=y
CONFIG_X86_MCE_P890_P892=y
CONFIG_X86_MCE_P893_P895=y
CONFIG_X86_MCE_P896_P898=y
CONFIG_X86_MCE_P899_P900=y
CONFIG_X86_MCE_P901_P903=y
CONFIG_X86_MCE_P904_P906=y
CONFIG_X86_MCE_P907_P909=y
CONFIG_X86_MCE_P910_P912=y
CONFIG_X86_MCE_P913_P915=y
CONFIG_X86_MCE_P916_P918=y
CONFIG_X86_MCE_P919_P920=y
CONFIG_X86_MCE_P921_P923=y
CONFIG_X86_MCE_P924_P926=y
CONFIG_X86_MCE_P927_P929=y
CONFIG_X86_MCE_P930_P932=y
CONFIG_X86_MCE_P933_P935=y
CONFIG_X86_MCE_P936_P938=y
CONFIG_X86_MCE_P939_P940=y
CONFIG_X86_MCE_P941_P943=y
CONFIG_X86_MCE_P944_P946=y
CONFIG_X86_MCE_P947_P949=y
CONFIG_X86_MCE_P950_P952=y
CONFIG_X86_MCE_P953_P955=y
CONFIG_X86_MCE_P956_P958=y
CONFIG_X86_MCE_P959_P960=y
CONFIG_X86_MCE_P961_P963=y
CONFIG_X86_MCE_P964_P966=y
CONFIG_X86_MCE_P967_P969=y
CONFIG_X86_MCE_P970_P972=y
CONFIG_X86_MCE_P973_P975=y
CONFIG_X86_MCE_P976_P978=y
CONFIG_X86_MCE_P979_P980=y
CONFIG_X86_MCE_P981_P983=y
CONFIG_X86_MCE_P984_P986=y
CONFIG_X86_MCE_P987_P989=y
CONFIG_X86_MCE_P990_P992=y
CONFIG_X86_MCE_P993_P995=y
CONFIG_X86_MCE_P996_P998=y
CONFIG_X86_MCE_P999_1000=y
```

编译源码

- `$ make modules` # 构建Linux内核模块

模块类型	描述
文件系统模块	文件系统是 Linux 内核的核心功能之一，内核中支持了很多不同的文件系统模块，例如 Ext2、Ext3、Ext4、Btrfs、XFS 等等。
网络模块	Linux 内核中支持很多网络模块，包括网络协议栈、设备驱动程序等等。常见的网络模块包括 TCP/IP 协议栈、UDP 协议栈、网络设备驱动程序等等。
设备驱动模块	Linux 内核支持很多不同类型的设备驱动模块，包括输入设备驱动、网络设备驱动、存储设备驱动、图形设备驱动等等。
加密模块	Linux 内核支持多种加密模块，包括用于数据加密和解密的模块、用于证书管理和验证的模块等等。
调试模块	Linux 内核支持很多不同类型的调试模块，包括用于内核调试的模块、用于用户空间程序调试的模块等等。
虚拟化模块	Linux 内核支持多种虚拟化技术，包括 KVM、Xen、VirtualBox 等等。每种虚拟化技术都需要对应的内核模块来实现。
安全模块	Linux 内核支持多种安全模块，包括 SELinux、AppArmor 等等。这些模块可以增强系统的安全性，限制进程的访问权限等等。

更新内核

- `$ make install`

安装编译好的内核。这个命令将会安装内核和其它相关文件到 `/boot` 目录中。



注意！！当我们想要更新系统内核时，一定要小心谨慎，不要误删或损坏现有的内核，否则你的电脑可能会抱怨它找不到自己的"大脑"了，然后就会陷入混乱之中。

更新内核（安全方式）

- `$ apt update`
- `$ apt install linux-image-generic`
`# Generic Linux kernel image`
- `$ reboot`

Linux 文件系统(FS) 源码

- 文件 file 的结构声明:

include/linux/fs.h

:852

f_path: 文件路径

inode: inode指针

f_mode: 文件模式

f_owner: 文件属主

f_mapping: 映射地址空间

```
852 struct file {
853     union {
854         struct llist_node    fu_llist;
855         struct rcu_head      fu_rcuhead;
856     } f_u;
857     struct path              f_path;
858     struct inode              *f_inode; /* cached value */
859     const struct file_operations *f_op;
860
861     /*
862      * Protects f_ep_links, f_flags.
863      * Must not be taken from IRQ context.
864      */
865     spinlock_t                f_lock;
866     atomic_long_t              f_count;
867     unsigned int               f_flags;
868     fmode_t                    f_mode;
869     struct mutex                f_pos_lock;
870     loff_t                     f_pos;
871     struct fown_struct          f_owner;
872     const struct cred           *f_cred;
873     struct file_ra_state        f_ra;
874
875     u64                         f_version;
876 #ifdef CONFIG_SECURITY
877     void                         *f_security;
878 #endif
879     /* needed for tty driver, and maybe others */
880     void                         *private_data;
881
882 #ifdef CONFIG_EPOLL
883     /* Used by fs/eventpoll.c to link all the hooks to this file */
884     struct list_head            f_ep_links;
885     struct list_head            f_tfile_llink;
886 #endif /* #ifdef CONFIG_EPOLL */
887     struct address_space        *f_mapping;
888 } __attribute__((aligned(4))); /* lest something weird decides that 2 is OK */
```

Linux 文件系统(FS) 源码

- `i_node` 的结构声明:

`include/linux/fs.h`

:591

`i_mode`: 文件模式

`i_sb`: 元数据信息
(`super_block`)

`i_atime`: access time

`i_mtime`: modify time

`i_ctime`: change time

```
591 struct inode {
592     umode_t          i_mode;
593     unsigned short   i_opflags;
594     kuid_t           i_uid;
595     kgid_t           i_gid;
596     unsigned int     i_flags;
597
598 #ifdef CONFIG_FS_POSIX_ACL
599     struct posix_acl  *i_acl;
600     struct posix_acl  *i_default_acl;
601 #endif
602
603     const struct inode_operations *i_op;
604     struct super_block *i_sb;
605     struct address_space *i_mapping;
606
607 #ifdef CONFIG_SECURITY
608     void *i_security;
609 #endif
610
611     /* Stat data, not accessed from path walking */
612     unsigned long i_ino;
613     /*
614      * Filesystems may only read i_nlink directly.
615      * following functions for modification:
616      *
617      * (set|clear|inc|drop)_nlink
618      * inode_(inc|dec)_link_count
619      */
620     union {
621         const unsigned int i_nlink;
622         unsigned int __i_nlink;
623     };
624     dev_t i_rdev;
625     loff_t i_size;
626     struct timespec i_atime;
627     struct timespec i_mtime;
```

f_mode vs. i_mode

- **f_mode** : 文件打开时的访问权限。

在 `struct file` 结构体中, `f_mode` 是一个 32 位的数字, 其中包括文件打开时的访问权限、文件打开的标志位等信息, 不包括文件类型信息。

- **i_mode** : 文件的实际权限。

在 `struct inode` 结构体中, `i_mode` 是一个 16 位的数字, 其中高 4 位表示文件类型, 低 12 位表示文件的访问权限。文件类型包括常见的文件类型如普通文件、目录、符号链接等, 访问权限包括读、写、执行等。

Linux 文件系统(FS) 源码

- `super_block` 的结构声明:

`include/linux/fs.h`
:1301

`s_dev`: 存储文件的设备号

`s_blocksize`: 数据块大小

`s_type`: 文件系统类型

`s_magic`: 幻数 / 魔数

`s_count`: 文件被链接的次数

```
1301 struct super_block {
1302     struct list_head      s_list;           /* Keep this first
1303     dev_t                 s_dev;           /* search index; _n
1304     unsigned char         s_blocksize_bits;
1305     unsigned long         s_blocksize;
1306     loff_t                s_maxbytes;      /* Max file size */
1307     struct file_system_type *s_type;
1308     const struct super_operations *s_op;
1309     const struct dquot_operations *dq_op;
1310     const struct quotactl_ops *s_qcop;
1311     const struct export_operations *s_export_op;
1312     unsigned long         s_flags;
1313     unsigned long         s_iflags;        /* internal SB_I*
1314     unsigned long         s_magic;
1315     struct dentry          *s_root;
1316     struct rw_semaphore   s_umount;
1317     int                   s_count;
1318     atomic_t              s_active;
1319     #ifdef CONFIG_SECURITY
1320     void                  *s_security;
1321     #endif
1322     const struct xattr_handler **s_xattr;
1323
1324     struct hlist_bl_head  s_anon;           /* anonymous dentri
1325     struct list_head      s_mounts;        /* list of mounts;
1326     struct block_device   *s_bdev;
1327     struct backing_dev_info *s_bdi;
1328     struct mtd_info       *s_mtd;
1329     struct hlist_node     s_instances;
1330     unsigned int          s_quota_types;   /* Bitmask of suppo
1331     struct quota_info     s_dquot;        /* Diskquota specif
1332
1333     struct sb_writers     s_writers;
1334
1335     char s_id[32];         /* Informational no
1336     u8 s_uuid[16];        /* UUID */
```

Linux 文件系统(FS) 源码

- 魔数的结构声明:

```
include/uapi/linux/  
magic.h
```

EXT4_SUPER_MAGIC: Ext4 普通文件

PIPEFS_MAGIC: 管道文件

USBDEVICE_SUPER_MAGIC: USB存储设备的
文件系统

```
#ifndef __LINUX_MAGIC_H__  
#define __LINUX_MAGIC_H__  
  
#define ADFS_SUPER_MAGIC      0xadf5  
#define ADFS_SUPER_MAGIC      0xadff  
#define AFS_SUPER_MAGIC      0x5346414F  
#define AUTOFS_SUPER_MAGIC    0x0187  
#define CODA_SUPER_MAGIC      0x73757245  
#define CRAMFS_MAGIC          0x28cd3d45    /* some random number */  
#define CRAMFS_MAGIC_WEND      0x453dcd28    /* magic number with the w  
#define DEBUGFS_MAGIC         0x64626720  
#define SECURITYFS_MAGIC       0x73636673  
#define SELINUX_MAGIC          0xf97cfff8c  
#define SMACK_MAGIC            0x43415d53    /* "SMAC" */  
#define RAMFS_MAGIC            0x858458f6    /* some random number */  
#define TMPFS_MAGIC            0x01021994  
#define HUGETLBFS_MAGIC        0x958458f6    /* some random number */  
#define SQUASHFS_MAGIC         0x73717368  
#define ECRYPTFS_SUPER_MAGIC    0xf15f  
#define EFS_SUPER_MAGIC        0x414A53  
#define EXT2_SUPER_MAGIC       0xEF53  
#define EXT3_SUPER_MAGIC       0xEF53  
#define XENFS_SUPER_MAGIC      0xabba1974  
#define EXT4_SUPER_MAGIC       0xEF53  
#define BTRFS_SUPER_MAGIC       0x9123683E  
#define NILFS_SUPER_MAGIC       0x3434  
#define F2FS_SUPER_MAGIC        0xF2F52010  
#define HPFS_SUPER_MAGIC        0xf995e849  
#define ISOFS_SUPER_MAGIC       0x9660  
#define JFFS2_SUPER_MAGIC       0x72b6  
#define PSTOREFS_MAGIC          0x6165676C  
#define EFIVARFS_MAGIC          0xde5e81e4  
#define OSTFS_SUPER_MAGIC       0x00c0ffe
```

Linux 文件系统(FS) 源码

- 魔数的位置声明:

```
include/linux/ext2_fs.h:29
```

```
24 /*
25  * Maximal count of links to a file
26  */
27 #define EXT2_LINK_MAX          32000
28
29 #define EXT2_SB_MAGIC_OFFSET    0x38
30 #define EXT2_SB_BLOCKS_OFFSET  0x04
31 #define EXT2_SB_BSIZE_OFFSET   0x18
32
```

Linux 文件系统(FS) 源码

- 如何在文件系统中指定, “.” 和 “..” 为当前和上一级目录?
 - **原理**: 在创建新的文件目录时, 需要初始化其对应的目录项数据。其中, . 指向该目录自身的 inode, 而 .. 目录项则指向其父目录的 inode。
 - VFS 定义 (fs/namei.c:3655) `vfs_mkdir`

```
(inode*)dir->i_op->mkdir(dir,dentry,mode)
```
 - VFS 调用各自文件系统所支持的 `mkdir` 函数来建立新的目录项

Linux 文件系统(FS) 源码

- 如何在文件系统中指定, “.” 和 “..” 为当前和上一级目录?

- Ext4 FS 定义 (fs/ext4/namei.c:2615) ext4_mkdir

2641: ext4_mkdir => ext4_init_new_dir

2571: ext4_init_new_dir => ext4_init_dot_dotdot

```
2543 struct ext4_dir_entry_2 *ext4_init_dot_dotdot(struct inode *inode,
2544 struct ext4_dir_entry_2 *de,
2545 int blocksz, int csum_size,
2546 unsigned int parent_ino, int dotdot_real_len)
2547 {
2548     de->inode = cpu_to_le32(inode->i_ino);
2549     de->name_len = 1;
2550     de->rec_len = ext4_rec_len_to_disk(EXT4_DIR_REC_LEN(de->name_len),
2551                                     blocksize);
2552     strcpy(de->name, ".");
2553     ext4_set_de_type(inode->i_sb, de, S_IFDIR);
2554
2555     de = ext4_next_entry(de, blocksize);
2556     de->inode = cpu_to_le32(parent_ino);
2557     de->name_len = 2;
2558     if (!dotdot_real_len)
2559         de->rec_len = ext4_rec_len_to_disk(blocksize -
2560                                             (csum_size + EXT4_DIR_REC_LEN(1)),
2561                                             blocksize);
2562     else
2563         de->rec_len = ext4_rec_len_to_disk(
2564             EXT4_DIR_REC_LEN(de->name_len), blocksize);
2565     strcpy(de->name, "..");
2566     ext4_set_de_type(inode->i_sb, de, S_IFDIR);
2567
2568     return ext4_next_entry(de, blocksize);
2569 }
```

辅助阅读服务

- <https://chat.openai.com/>
- <https://elixir.bootlin.com/linux/latest/source>

静若处子
动如脱兔

